FUNCTIONAL REQUIREMENTS

Toy Haven

Integrative Programming and Technology I

DE LEON, JOSEPH BENJAMIN U.
ESTANES, CHARLIE JR. C.
BSIT - S - 3A

Description:

In this project, we plan to make a web application using React, MongoDB, & Node.js (w/ Express) for a made-up toy store called “Toy Haven”, targeted at kids, parents, and toy collectors alike. It will show the usual e-commerce style website to demonstrate our knowledge of the M.E.R.N. stack. The store sells six kinds of products: Lego sets, RC cars, Hot Wheels, Nerf blasters, puzzles, and plush toys. The goal is to make the website run with no errors and to give customers an easy way to find, order, and review their favorite toys.

To make it easier to understand the schema of the project, we’ll be showing the collections that will be made for the system. Each collection will have its own purpose and will store the information needed for the website to function properly.

The main collections of the project will include **Users, Categories, Products, Carts, Orders, Payments, and Reviews**. The **Users** collection will store customer information such as their name, email, password, and account details. The **Products** collection will contain information about the toys available in the store, including the product name, category, description, photos, and variants with their own price and stock.

The **Orders** collection will be used to keep track of the products purchased by customers, including the customer who placed the order, the items ordered (with a snapshot of the name and price at the time of purchase), the order status, and the date of purchase. The **Reviews** collection will allow customers to leave ratings and comments about the products they have purchased. Lastly, the **Categories** collection will organize the products into the six available toy categories: **Lego sets, RC cars, Hot Wheels, Nerf blasters, puzzles, and plush toys.**

These collections will be connected through ObjectId references where necessary to allow the system to properly manage users, products, orders, and reviews. The React frontend will provide the interface that customers interact with, while Node.js and Express will handle the backend logic and API requests. MongoDB will be used to store and manage the application's data. Through the use of these technologies, the project aims to demonstrate how the M.E.R.N. stack can be used to create a functional and user-friendly e-commerce web application.

Features:

- Two authentication roles: Administrators and Customers (with login/authorization & verify registration using required mailer)
- Frontend (React) & Backend (Node.js – Express.js)
- NoSQL Database (MongoDB)
- Product categories: Lego Sets, RC Cars, Hot Wheels, Nerf Blasters, Puzzles, and Plush Toys.
- Variations will be added for products such as color, size, or set edition.
- Returns, Cancellations, and Reviews will be added.
- Packages used: multer, Cloudinary, nodemailer, express-validator, bad-words, pdfkit, and recharts.

Detailed Features:

1. User Management
Users can register, log in, and log out
Users must verify their email before logging in
Users can edit their personal details
Users can upload and update profile photos
Admin can view, deactivate, or delete user accounts (delete is a soft delete)
Admin can update user roles

2. Product Management
Admin can add, edit, or delete products (delete is a soft delete, so past orders keep their data)
Users can view products
Users can search products by name or category
Admin can upload and update product photos

3. Category and Variant Management
Admin can add the six categories, edit them, or delete them (delete is a soft delete)
Users can browse products by category
Admin can add, edit, delete, or view product variants (label, price, discountPercent, stock)
Admin can update variant stock

4. Shopping Cart and Order Management
Users can add and remove items to the cart
Users can update cart quantity or selected variant
Users can view the cart
Users can place orders; stock is reduced with an atomic update so it can never go below zero
Each order item stores the product name, variant, and unit price at the time of purchase
Users can edit their order (shipping info only) while the status is pending
Users can cancel an order only while the status is pending or processing; stock is restored on cancel
Users can request a return only after the order is delivered; Admin approves or rejects it
Users can view their order history
Admin can update order status and view all orders
Order status: pending, processing, shipped, delivered, cancelled, returned

5. Payment Management and Reports
Users can select a payment method
System can process and confirm payments
Admin can generate sales reports

6. Search and Validation
Users can search and filter products easily
System validates inputs and prevents duplicates

7. Review and Rating
Customers can submit reviews for purchased products only
Only one review per user per product
Users can edit or delete their reviews
Users can view product reviews
System filters or masks foul words in reviews

Proposed Collections:

Collection	Main fields
users	name, email, password (hashed), role (admin/customer), profilePhoto, isVerified, isActive, isDeleted, timestamps
categories	name (Lego Sets, RC Cars, Hot Wheels, Nerf Blasters, Puzzles, Plush Toys), description, isDeleted, timestamps
products	name, description, category (ObjectId ref), photos, variants (label, price, discountPercent, stock), isDeleted, timestamps
carts	user, items (product, variant, quantity), timestamps
orders	user, items (product, variant, name, unitPrice, quantity), shippingInfo, status (pending/processing/shipped/delivered/cancelled/returned), paymentMethod, returnRequest (reason, status, requestedAt), timestamps
payments	order, method, amount, status (pending/confirmed/failed), paidAt, timestamps
reviews	user, product, order, rating, comment, timestamps (unique index on user + product)